



Amrita School of AI
AMRITA VISHWA VIDYAPEETHAM, COIMBATORE

20% OF THE COURSE

Mini Project Catalogue

Choose one problem, parallelise it properly, and defend the result with evidence. The engineering is worth more than the speedup.

Prepared by **Dr. Abhijith Anandakrishnan**

Assistant Professor, Amrita School of AI

WEIGHT

20% of the course

MARKS

50

TEAM SIZE

2 students

PROPOSED

Week 8

PRESENTED

Final week

OFFERING

Odd Semester 2026-27

What is being assessed

Not how fast your program is. Whether you can:

1. Establish what the ceiling is before optimising.
2. Choose a parallel decomposition and justify it.
3. Implement it correctly, including the concurrency correctness.
4. Measure honestly, with a scaling study that could be reproduced.
5. Explain the gap between what you achieved and what was attainable.

A project that reaches 40 percent of the roofline and explains precisely why it stopped there scores higher than one that reaches 60 percent by accident and cannot account for it.

Requirements, whichever project you choose

- A correct **serial baseline**, optimised, used as T_1 .
- At least **two** parallel technologies: OpenMP, MPI, CUDA, or a hybrid.
- A **strong-scaling study** on `asaicompute` with thread pinning, and efficiency and Karp-Flatt reported at every point.
- A **roofline analysis** of the dominant kernel: predicted ceiling, achieved performance, and the reason for the gap.
- **Correctness evidence** for the parallel version: not “it printed the same number once”, but a repeated comparison across thread and rank counts.
- A **reproducibility statement**: machine, compiler, flags, sizes, repetitions, statistic.

PITFALL · THE TWO FAILURE MODES WE SEE EVERY YEAR

No baseline. A speedup measured against your own one-thread parallel run, or against an unoptimised build, is not a speedup. Fix the serial version first; in Unit 2’s worked example that alone was worth 13.5x.

A single correctness check. Concurrent code that produced the right answer once has not been shown to be correct. Run it repeatedly across thread counts and compare, exactly as the exercise grader does.

The catalogue

Twelve options. Pick one, or propose your own in week 8.

A. Classical simulation

A1. 2-D heat diffusion with halo exchange. Extend the Unit 2 stencil to two dimensions. Compare 1-D strip and 2-D block decomposition and confirm the surface-to-volume prediction ($4\sqrt{P}/N$ against $2P/N$) against measurement. Hybrid MPI plus OpenMP for the final version. *Hardest part: making the 2-D decomposition's corner exchanges correct.*

A2. N-body gravitational simulation. Direct $O(N^2)$ force summation, then a cut-off with a neighbour list. OpenMP, then CUDA. Discuss why the naive version is compute bound and what the cut-off does to the arithmetic intensity. *Hardest part: load imbalance once the cut-off makes work per particle uneven.*

A3. Conjugate gradient solver for a sparse system. Discretise a Poisson problem, solve with CG. Each iteration is a sparse matrix-vector product plus two dot products. MPI, with halo exchange and `MPI_Allreduce`. Report how the two global reductions come to dominate as rank count grows. *Hardest part: sparse storage format and its effect on coalescing or vectorisation.*

A4. Lattice Boltzmann fluid flow. A stencil with a large state per cell. Excellent for demonstrating that data layout, array-of-structures against structure-of-arrays, changes performance more than any amount of loop tuning. *Hardest part: the layout experiment is the project; do not skip it.*

B. Data and analytics

B5. Parallel k-means clustering. OpenMP then MPI. The assignment step is embarrassingly parallel; the update step is a reduction. Study how cluster count and dimensionality move the kernel along the roofline. *Hardest part: a correct distributed centroid update without accumulating error.*

B6. Parallel sorting. Implement and compare two of: sample sort, bitonic sort, radix sort. MPI for distributed, CUDA for on-device. Compare against `std::sort` and against Thrust. *Hardest part: the all-to-all redistribution in sample sort.*

B7. Graph breadth-first search. Level-synchronous BFS on a large graph. Try it on GPU and explain, with profiler evidence, why it performs badly: divergence, uncoalesced access, low intensity. *Hardest part: producing an honest negative result, which is the point of this one.*

C. GPU-centred

C8. Matrix multiplication, all the way. Naive, tiled, register-blocked, then cuBLAS. Reproduce the measured table from Unit 3 on your allocated GPU and explain each step's contribution with Nsight Compute evidence. Include a tile-size sweep. *Hardest part: register blocking without destroying occupancy.*

C9. 2-D convolution / image filtering. Separable and non-separable filters, constant memory for coefficients, shared memory for tiles, halo handling at tile edges. Compare against cuDNN. *Hardest part: correct halos at block boundaries.*

C10. Monte Carlo option pricing or integration. Embarrassingly parallel, so the interest is elsewhere: random number generation per thread (cuRAND), reduction of results, and a genuine study of numerical reproducibility across thread counts. *Hardest part: making the RNG streams independent and the result reproducible.*

D. HPC and AI

D11. Data-parallel training from scratch. Train a small network on two GPUs with an all-reduce you implement yourself, then with NCCL. Measure the communication-to-computation ratio and show where it starts to dominate. Relate it to the Unit 4 arithmetic. *Hardest part: overlapping the gradient reduction with the backward pass.*

D12. Inference optimisation. Take a small transformer. Measure single-token generation and show it is memory bound. Then improve throughput with batching, and quantify how batch size moves the kernel along the roofline. Optionally add quantisation. *Hardest part: measuring cleanly enough that the roofline argument is convincing.*

Deliverables

ITEM	WHEN
One - page proposal: problem, baseline plan, decomposition, expected bottleneck	Week 8
Source repository, with a README that builds and runs it	Final week
Report, 8 to 12 pages	Final week
Presentation, 10 minutes plus 5 of questions	Final week

The report must contain: problem statement, serial baseline and its optimisation, parallel design with justification, correctness evidence, scaling study with plots, roofline analysis, and an honest limitations section.

Marking

MARKS	FOR
8	Serial baseline: correct, optimised, and justified as the right T_1
10	Parallel design: decomposition, granularity and load balance, argued not assumed
8	Correctness: repeated verification across thread and rank counts
10	Measurement: scaling study, efficiency, Karp-Flatt, reproducibility statement
8	Roofline analysis: predicted ceiling, achieved, and the gap explained
6	Report and presentation: clear, honest, and answers questions

NOTE · NEGATIVE RESULTS SCORE FULL MARKS

Project B7 exists to make this explicit. If you show, with profiler evidence, that a GPU is the wrong tool for your problem and explain exactly why, that is a complete and successful project. Knowing when not to use an accelerator is worth as much as knowing how.

PITFALL · WHAT LOSES MARKS

Speedup against a bad baseline. A single correctness run. Plots without axis labels or units. Timings with no machine or compiler stated. Claiming a percentage of peak without the roofline that makes it meaningful. Reporting a mean instead of a minimum without saying why.

Team working

Pairs. Both members present, and both answer questions on any part of the work. The report names who did what. A submission where one member cannot explain the design is marked accordingly for that member.